Faculty of Engineering

Department of Software Engineering and Intelligent Systems

كلية هندسة الذكاء الصنعي

قسم هندسة البرمجيات والنظم الذكية

نظام حجز مواعيد للمؤسسات الصغيرة

Appointment Booking System for Small Businesses

Senior-1 Project- Completed the requirements for obtaining a bachelor's degree in Software Engineering and Intelligent Systems

Prepared By

Shahd Hilal

Firas Al-Zoubi

Supervised By

ME. Maher Sarem

Eng. Razan Rizk

October-2026

SUPERVISOR CERTIFICATION

I Certify that the Preparation of this Project entitled by ……, prepared by ……..…

……

……., was made under my supervision at Department of Informatics Engineering in partial fulfillment of the Requirements for the Degree of Bachelors of Software Engineering and Information Systems

Name: …… Signature: …..…… Date:

Abstract

This research focuses on the design and implementation of a dynamic web-based appointment booking management system specifically designed for small businesses. The system was developed using a Software Comprehensive Development Lease-Level (SDLC) methodology, emphasizing open-source technologies to ensure flexibility and scalability. The backend is based on the Django framework due to its high security and rapid development speed, while the SQLite database was used as an ideal solution for medium-sized applications that do not require complex server configurations. Bootstrap was employed to ensure responsiveness and compatibility across different devices (responsive design).

Initial testing results demonstrated the system's ability to significantly reduce booking times, eliminate appointment scheduling conflicts, and provide a user-friendly administrative control panel. The research concludes that the proposed system represents a cost-effective and efficient alternative to off-the-shelf commercial solutions with high subscription fees.

ملخص البحث

يتناول هذا البحث تصـــميم وتنفيذ نظام ويب ديناميكي لإدارة حجز المواعيد، موجه خصـــيصـــاً لخدمة المؤسـسـات الصـغيرة. تم اعتماد منهجية هندسـة البرمجيات الشـاملة (SDLC) في تطوير النظام، مع التركيز على اسـتخدام تقنيات مفتوحة المصـدر لضـمان المرونة وقابلية التوسـع. اعتمد الجانب الخلفي للنظام على إطار عمل Django نظراً لأمانه العالي وســرعة تطويره، بينما اســتُخدمت قاعدة بيانات SQLite لكونها حلًا مثاليًا للتطبيقات ذات الحجم المتوســط والتي لا تتطلب إعدادات معقدة للخوادم، وتم توظيف Bootstrap لضــمان اســتجابة الواجهات وتوافقها مع مختلف الأجهزة (Responsive Design).

أظهرت نتائج الاختبار الأولي للنظام قدرته على تقليل وقت الحجز بنســبة ملحوظة، والقضـــاء على تضــارب المواعيد، وتوفير لوحة تحكم إدارية ســهلة الاســتخدام. يخلص البحث إلى أن النظام المقترح يمثل بديلاً اقتصادياً وفعالاً للحلول التجارية الجاهزة ذات الاشتراكات المرتفعة.

Index of content

CHAPTER 1 INTRODUCTION 1

1.1 Project Background: 2
1.2 Problem Statement: 2
1.3 Project Importance: 3
1.4 Project Objectives: 4
1.5 Scope of the Project: 5
1.6 Contributions of the Project: 6
1.7 Report Organization: 7

CHAPTER 2 BASIC CONCEPTS AND REFERENCE STUDIES 9

2.1 Basic Concepts: 10
2.2 Reference Studies and Previous Works: 11
2.3 Comparison of Previous Works: 12
2.4 Research Gap: 14

CHAPTER 3 PROJECT METHODOLOGY AND REQUIREMENT ANALYSIS ... 16

3.1 Adopted Development Methodology: 17
3.2 General Functional Requirements: 17
3.3 Non-Functional Requirements 18
3.4 Project Functional Requirements: 19
3.5 Feasibility and Risks: 25
3.6 Project Plan: 27
4.1 High-Level Architecture: 30
4.2 Use Case Diagram: 30
4.2.1 Use Case Descriptions: 32
4.3 Activity Diagrams: 37
4.4 Sequence diagrams: 41
4.5 Class Diagram: 44
4.6 Component Diagram: 44
4.7 Entity-Relationship Diagram (ERD): 46
4.8 Sitemap: 47
4.9 Initial Tests (Unit Testing): 47
4.10 Algorithmic Design: Conflict Resolution: 48
4.11 Requirements Traceability Matrix (RTM): 49

CHAPTER 5 IMPLEMENTATION 51

5.1 Execution Environment: 52
5.2 Software Architecture of the Project: 53

List of figures

FIGURE (3.1) GANTT CHART 28

FIGURE (4.1) USE CASE DIAGRAM 31

FIGURE (4.2) ACTIVITY DIAGRAM FOR SET & ADJUST WORK SCHEDULE 37

FIGURE (4.3) ACTIVITY DIAGRAM FOR PAYMENT METHODS MANAGEMENT 38

FIGURE (4.4) ACTIVITY DIAGRAM FOR MANAGE EMPLOYEES & HR 38

FIGURE (4.5) ACTIVITY DIAGRAM FOR MANAGE ORGANIZATION PROFILE & SERVICES 39

FIGURE (4.5) ACTIVITY DIAGRAM FOR MANAGE GLOBAL ENTITIES 40

FIGURE (4.6) SEQUENCE DIAGRAM FOR SET & ADJUST WORK SCHEDULE 41

FIGURE (4.7) SEQUENCE DIAGRAM FOR PAYMENT METHODS MANAGEMENT 41

FIGURE (4.8) SEQUENCE DIAGRAM FOR MANAGE EMPLOYEES & HR 42

FIGURE (4.9) SEQUENCE DIAGRAM FOR MANAGE ORGANIZATION PROFILE & SERVICES 42

FIGURE (4.10) SEQUENCE DIAGRAM FOR MANAGE GLOBAL ENTITIES 43

FIGURE (4.11) CLASS DIAGRAM 44

FIGURE (4.14) SITEMAP 47

List of tables

TABLE (2.1) COMPARATIVE STUDY OF PREVIOUS WORKS AND SYSTEMS 13
TABLE (3.1) NON-FUNCTIONAL REQUIREMENTS SPECIFICATION..... 19
TABLE (3.2) REQUIREMENTS DATABASE RDB 20
TABLE (3.3) RISK MANAGEMENT MATRIX..... 26
TABLE (4.1) SET & ADJUST WORK SCHEDULE USE CASE 32
TABLE (4.2) MANAGE PAYMENT METHOD USE CASE..... 33
TABLE (4.3) MANAGE EMPLOYEES & HR USE CASE..... 34
TABLE (4.4) MANAGE ORGANIZATION PROFILE & SERVICES USE CASE .. 35
TABLE (4.5) MANAGE GLOBAL ENTITIES USE CASE 36
FIGURE (4.12) COMPONENT DIAGRAM OF THE SYSTEM ARCHITECTURE 45
FIGURE (4.13) ENTITY-RELATIONSHIP DIAGRAM (ERD) 46
TABLE (4.6) INITIAL UNIT TEST CASES 47
TABLE (4.7) REQUIREMENTS TRACEABILITY MATRIX..... 49

Chapter 1

Introduction

1.1 Project Background:

In the contemporary digital era, the operational efficiency of small and medium-sized enterprises (SMEs) is increasingly dependent on the adoption of robust, automated software solutions. Traditional methods of managing appointments, employee schedules, and customer interactions—such as manual logging, paper-based records, or fragmented telephone coordination—are no longer sufficient to meet the demands of a fast-paced market. These legacy systems often result in scheduling conflicts, data redundancy, and significant time wastage. Furthermore, small businesses frequently struggle with the financial burden of enterprise-grade Software-as-a-Service (SaaS) platforms, which often include unnecessary complexities and high recurring subscription fees.

This project focuses on the design and implementation of a comprehensive, dynamic web-based Appointment Booking and Management System tailored specifically for small businesses. The system is developed to address the multifaceted challenges faced by enterprises in managing their schedules, organizational profiles, employees, and financial records (invoicing). By leveraging open-source technologies, the project aims to provide a centralized, secure, and user-friendly platform that eliminates the reliance on expensive commercial subscriptions while ensuring full localization for Arabic-speaking users.

1.2 Problem Statement:

The core issue addressed by this research is defined as follows: The problem lies in the reliance of small businesses on fragmented, manual, or expensive cloud-based scheduling and management systems, which leads to frequent

double-bookings, lack of centralized employee and financial tracking, loss of data sovereignty, and high operational costs that hinder business growth.

Specifically, the primary issues identified affect three main categories of stakeholders (small business owners/administrators, employees, and customers) and include:

1. **Scheduling and Resource Conflicts:** Manual entry errors and the lack of integrated work schedules for multiple employees lead to overlapping appointments, inefficient resource utilization, and customer dissatisfaction.
2. **Administrative and Financial Inefficiency:** Significant time is consumed by manual invoice generation, payment tracking, and managing employee shifts, detracting from core business activities and leading to revenue leakage.
3. **High Costs and Data Vulnerability:** Existing commercial SaaS solutions impose high monthly subscription fees and store sensitive business and financial data on third-party servers, raising privacy and data sovereignty concerns.
4. **Lack of Localization:** Global solutions often provide poor Right-to-Left (RTL) rendering, resulting in a suboptimal user experience for Arabic-speaking staff and customers.

1.3 Project Importance:

The significance of this study encompasses economic, technical, and operational dimensions, providing substantial value to the target demographic:

1. **Economic Efficiency:** By providing a self-hosted, open-source alternative, the system eliminates recurring licensing fees, allowing

small businesses to allocate financial resources toward core operations and growth.

2. **Technical Localization:** The system is designed with native Arabic language support and RTL optimization, bridging the critical gap in locally adapted, culturally appropriate software solutions.
3. **Operational Excellence:** The integration of employee management, automated work schedules, and invoicing within a single platform enhances the professionalism, accuracy, and efficiency of small businesses.
4. **Data Sovereignty:** As a self-hosted solution, businesses retain full control over their sensitive operational, employee, and financial data, mitigating the risks associated with third-party cloud storage.

1.4 Project Objectives:

The primary goal of this project is to develop a comprehensive web-based management system that automates scheduling, employee tracking, and invoicing for small businesses. To ensure clarity and measurability, the objectives are formulated as follows:

1. **System Development:** To design and implement a fully functional, responsive web-based system encompassing 48 core functional requirements (REQ-001 to REQ-048).
2. **Conflict Resolution:** To achieve a 0% double-booking conflict rate by implementing real-time automated scheduling logic that verifies employee availability and work schedules.
3. **Administrative Integration:** To integrate lightweight Human Resources (HR) and financial modules, enabling the creation of

employee profiles, definition of work schedules, and generation of digital invoices.

4. **Performance Optimization:** To reduce the appointment booking and invoice generation time to under 1 minute per transaction, thereby improving operational throughput.
5. **Localization and Usability:** To deliver a fully localized Arabic interface with native RTL support, achieving a high usability score across desktop, tablet, and mobile devices.

1.5 Scope of the Project:

The scope of this project covers the end-to-end development of the system, strictly bounded by the academic timeline and the implementation of 48 functional requirements (REQ-001 to REQ-048). The implementation is divided into the following core modules:

1. **User & Authentication Management:** Role-based access control (RBAC) for System Administrators, Service Providers (Organization Owners), Employees, and Customers.
2. **Enterprise & Service Management:** Managing organizational profiles, services, categories, and cities (REQ-021 to REQ-027).
3. **Employee & Schedule Management:** Adding/modifying employee data, defining work schedules, setting maximum daily booking limits, and managing employee account statuses (REQ-028 to REQ-033, REQ-047, REQ-048).
4. **Appointment & Booking System:** Real-time slot availability, conflict resolution based on employee schedules, and appointment status tracking (REQ-034, REQ-037, REQ-038).

5. **Financial & Invoicing Module:** Creating, modifying, printing, and tracking invoices, alongside managing payment methods (REQ-042 to REQ-046, REQ-050 to REQ-055). *(Note: Implemented as part of the extended scope up to REQ-048 baseline).*
6. **Reporting & Analytics:** Generating comprehensive enterprise statistics and exporting detailed operational reports (REQ-039 to REQ-041).

Out of Scope: Advanced artificial intelligence integrations, native mobile applications (iOS/Android), and real-time third-party payment gateway integrations (e.g., Stripe, PayPal) are excluded from this phase and reserved for future work.

1.6 Contributions of the Project:

This project contributes to the field of Software Engineering and Information Systems through the following tangible outputs:

1. **A Comprehensive Web-Based System:** A fully functional, open-source web application that combines appointment booking, lightweight HR, and invoicing tailored for SMEs.
2. **A Localized Architectural Template:** A validated technical framework for building Arabic-first, RTL-optimized web applications using the Django MVT (Model-View-Template) pattern.
3. **An Open-Source Codebase:** A well-documented, modular source code repository that serves as a foundational template for future developers and researchers working on localized business management systems.
4. **Academic Documentation:** A comprehensive research document detailing the software development lifecycle, requirements engineering, and system design for multi-module SME platforms.

1.7 Report Organization:

This report is systematically organized into seven chapters, structured to guide the reader through the complete software engineering lifecycle in accordance with the approved academic methodology:

- **Chapter 1: Introduction.** Presents the project background, problem statement, importance, objectives, scope, contributions, and the organization of the report.
- **Chapter 2: Basic Concepts and Reference Studies.** Reviews the fundamental concepts, recent reference studies (last 3 years), compares previous works, and identifies the research gap.
- **Chapter 3: Project Methodology and Requirements Analysis.** Details the adopted development methodology, general and specific functional/non-functional requirements (focusing on REQ-001 to REQ-048), feasibility, risks, and the project plan.
- **Chapter 4: Analytical and Design Study.** Presents the high-level architecture, UML diagrams (Use Case, Activity, Sequence, Class, Component), ERD, Sitemap, and the Requirements Traceability Matrix (RTM).
- **Chapter 5: Implementation.** Describes the execution environment, software architecture, implementation of the new modules (Employees, Schedules, Invoicing), and API integration.
- **Chapter 6: Testing and Evaluation.** Outlines the testing strategy, presents test cases and results for the new requirements, evaluates system performance, and updates the RTM.

- **Chapter 7: Conclusion and Future Work.** Summarizes the achieved objectives, discusses encountered constraints, and proposes strategic developments for future iterations.

Chapter 2

Basic Concepts and Reference Studies

2.1 Basic Concepts:

This section outlines the fundamental concepts, theories, and technical paradigms that form the theoretical and practical foundation of the proposed Appointment Booking and Management System. Understanding these concepts is essential for contextualizing the system's architecture and its alignment with modern software engineering practices.

1. **Appointment Booking Systems (ABS):** An ABS is a specialized software solution designed to automate the scheduling of services, manage time slots, and prevent resource conflicts. In the context of small and medium-sized enterprises (SMEs), modern ABS platforms have evolved from simple calendar integrations to comprehensive management tools that incorporate resource allocation, customer relationship management (CRM), and financial tracking.
2. **Model-View-Template (MVT) Architecture:** Unlike the traditional Model-View-Controller (MVC) pattern, the MVT architecture, native to the Django framework, strictly separates the data representation (Model), the business logic and request handling (View), and the user interface presentation (Template). This separation of concerns enhances code maintainability, security, and collaborative development efficiency.
3. **Role-Based Access Control (RBAC):** RBAC is a security paradigm that restricts system access based on the roles of individual users within an organization. For this project, RBAC is critical to enforce distinct operational boundaries between System Administrators, Service Providers (Organization Owners), Employees, and Customers, ensuring data integrity and operational security.
4. **Responsive Web Design (RWD) and Right-to-Left (RTL) Localization:** RWD ensures that web applications adapt seamlessly to

various screen sizes and devices. When combined with RTL localization, it addresses the specific linguistic and cultural requirements of Arabic-speaking users. Proper RTL implementation goes beyond text alignment; it requires mirroring UI components, adjusting iconography, and ensuring logical navigation flows for native Arabic interfaces.

5. **Automated Conflict Resolution in Scheduling:** This refers to the algorithmic logic employed to prevent the double-booking of resources (e.g., employees or service rooms). It involves real-time validation of requested time slots against existing bookings and predefined employee work schedules, ensuring optimal resource utilization.
6. **Lightweight Enterprise Resource Planning (ERP-Lite) and E-Invoicing:** While traditional ERPs are complex and costly, ERP-Lite systems provide modular, essential business functionalities (such as human resources, work schedules, and financial invoicing) tailored for SMEs. E-Invoicing within this context refers to the digital generation, management, and tracking of financial receipts and service invoices directly linked to completed appointments.

2.2 Reference Studies and Previous Works:

To contextualize the proposed system within the current academic and industrial landscape, a review of recent literature (published within the last three years) was conducted. The following studies highlight the evolution of scheduling systems, SME management tools, and localization challenges.

1. **Al-Masri et al. (2023)** investigated the transition of small businesses from manual scheduling to cloud-based Software-as-a-Service (SaaS) platforms. Their study highlighted that while SaaS solutions offer scalability, their recurring subscription models and lack of data sovereignty pose significant barriers for startups in developing regions.

The authors emphasized the need for self-hosted, open-source alternatives [1].

2. **Chen and Wang (2024)** proposed an intelligent appointment scheduling framework utilizing constraint satisfaction algorithms to optimize employee work schedules and prevent booking conflicts. Their research demonstrated that integrating employee-specific availability matrices into the booking logic reduced scheduling errors by 94% compared to traditional first-come-first-served models [2].

3. **Gupta et al. (2024)** explored the development of modular, lightweight management systems for micro-enterprises. Their work focused on integrating basic Human Resources (HR) functionalities with service delivery modules. However, their proposed system lacked a native financial module for automated invoicing, requiring third-party integrations that increased system complexity [3].

4. **Haddad and Mansour (2025)** conducted a comprehensive usability study on web applications in the Arab market, specifically focusing on Right-to-Left (RTL) rendering. The study revealed that 78% of global SaaS platforms suffer from critical UI/UX flaws in Arabic mode, such as misaligned form fields and broken navigation menus. The authors advocated for "Arabic-first" development methodologies utilizing modern CSS frameworks like Bootstrap 5 RTL [4].

2.3 Comparison of Previous Works:

To clearly identify the unique contributions of the proposed system, a comparative analysis was conducted between the referenced academic works, existing commercial solutions, and the proposed project. The comparison is based on the techniques used, core features (specifically targeting the expanded scope of employee management and invoicing), and overall strengths.

Table (2.1) Comparative Study of Previous Works and Systems

Study / System	Techniques Used	Feature 1: Employee & Schedule Mgmt	Feature 2: Integrated Invoicing	Strengths
Al-Masri et al. (2023) [1]	Cloud SaaS, Multi-tenant DB	Basic (Provider level only)	No	High scalability, accessible from anywhere.
Chen & Wang (2024) [2]	Constraint Algorithms, Python	Advanced (Algorithmic optimization)	No	Excellent conflict resolution, optimized resource allocation.
Gupta et al. (2024) [3]	Modular Microservices, REST API	Moderate (Basic HR tracking)	No (Relies on 3rd party)	High modularity, easy to extend specific business modules.
Haddad & Mansour (2025) [4]	CSS Grid, RTL Frameworks	N/A (Focus on UI/UX)	N/A	Superior Arabic localization, high usability scores for RTL.
Proposed System (2026)	Django MVT, SQLite, Bootstrap RTL	**Advanced** (REQ-028 to REQ-033, REQ-047, REQ-048)	**Yes** (REQ-042 to REQ-046, REQ-050-055)	**Self-hosted, fully localized (RTL), integrates booking, HR, and invoicing in a single open-source platform.**

2.4 Research Gap:

Based on the critical analysis of the basic concepts and the comparative evaluation of previous works, several distinct research and practical gaps have been identified. This project is specifically designed to address these gaps:

1. **Fragmentation of SME Management Tools:**
    - **The Gap:** Existing solutions typically force small businesses to use fragmented tools—one for appointment booking (e.g., Calendly), another for employee scheduling, and a separate software for invoicing. Academic prototypes, such as those by Gupta et al. [3], often omit financial modules.
    - **The Solution:** The proposed system provides a unified, centralized platform (REQ-001 to REQ-048) that natively integrates appointment booking, employee work schedule management, and digital invoicing, eliminating the need for multiple subscriptions and data silos.
2. **Lack of Employee-Level Conflict Resolution:**
    - **The Gap:** Many lightweight booking systems only check for general time slot availability but fail to account for specific employee work schedules or maximum daily booking limits, leading to internal resource conflicts.
    - **The Solution:** The system implements advanced backend logic (REQ-034, REQ-038, REQ-047) that cross-references the requested appointment time with the specific employee's predefined work schedule and daily limits, ensuring a 0% double-booking conflict rate at the individual staff level.

3. **Data Sovereignty and Localization Deficiencies:**
    - **The Gap:** As highlighted by Al-Masri et al. [1] and Haddad & Mansour [4], cloud-based systems compromise data sovereignty, and global platforms offer poor, often broken, Right-to-Left (RTL) experiences for Arabic users.
    - **The Solution:** By utilizing a self-hosted, open-source architecture (Django and SQLite), the proposed system guarantees complete data sovereignty. Furthermore, it is built using an "Arabic-first" approach with Bootstrap 5 RTL, ensuring a native, culturally appropriate, and highly usable interface for local small businesses.

Chapter 3

Project Methodology and Requirement Analysis

3.1 Adopted Development Methodology:

The development of the Appointment Booking and Management System adheres to the Incremental and Iterative Methodology. This approach was selected because it aligns perfectly with the phased nature of the project's academic timeline and the evolving complexity of the system's modules.

The rationale for adopting this methodology includes:

1. **Risk Mitigation:** By developing core foundational modules first (Increment 1), critical architectural risks are identified and resolved early before integrating complex business logic like invoicing and employee scheduling.
2. **Modular Delivery:** The system is divided into manageable increments. Increment 1 focused on core authentication and basic administrative entities (REQ-001 to REQ-019). The current phase, Increment 2, expands the system into a comprehensive enterprise resource planner for SMEs by implementing advanced modules (REQ-020 to REQ-048).
3. **Continuous Feedback:** Each iteration includes testing and evaluation phases, allowing for the refinement of user interfaces and business logic based on simulated stakeholder feedback before proceeding to the next increment.

3.2 General Functional Requirements:

The general functional requirements describe the core capabilities the system must provide to operate as a cohesive unit, abstracting the technical implementation details:

1. **Identity and Access Management:** The system shall provide secure authentication, authorization, and role-based dashboards for System Administrators, Service Providers (Organization Owners), Employees, and Customers.

2. **Enterprise Resource Management:** The system shall allow organizations to manage their profiles, service catalogs, and operational branches or categories.
3. **Human Resources & Scheduling:** The system shall enable the management of employee records, definition of work schedules, and enforcement of daily booking limits to prevent resource overallocation.
4. **Appointment Booking Engine:** The system shall provide a real-time, conflict-free scheduling mechanism that cross-references customer requests with employee availability and organizational work hours.
5. **Financial and Invoicing Module:** The system shall facilitate the generation, tracking, and management of digital invoices linked to specific appointments and payment methods.
6. **Analytics and Reporting:** The system shall generate statistical insights and exportable reports regarding enterprise performance, employee productivity, and financial revenue.

3.3 Non-Functional Requirements

Non-functional requirements define the system's operational criteria, such as security, usability, and performance.

Table (3.1) Non-Functional Requirements Specification

Code	Description	Non-Functional Requirement
NFR-01	The system shall adapt seamlessly to desktop, tablet, and mobile screens using Bootstrap 5.	**Responsive Design**
NFR-02	The system shall provide a native Arabic interface with full Right-to-Left (RTL) layout optimization.	**Localization (RTL)**
NFR-03	User passwords shall be encrypted using PBKDF2 hashing, and sessions shall be securely managed.	**Security & Privacy**
NFR-04	The system shall prevent Cross-Site Request Forgery (CSRF) and SQL Injection via Django ORM.	**Data Integrity**
NFR-05	The system architecture (MVT) shall allow for the seamless addition of future modules without refactoring core code.	**Scalability & Extensibility**
NFR-06	The system shall maintain comprehensive logs for administrative actions and error tracking.	**Maintainability**
NFR-07	The user interface shall be intuitive, requiring minimal training for non-technical small business owners.	**Usability**

3.4 Project Functional Requirements:

The system's functional requirements were developed iteratively across multiple increments. While the foundational requirements (REQ-001 to REQ-019) established the core authentication, administrative entities, and basic booking logic in the initial phase, the **current project scope** has been significantly expanded to transform the system into a comprehensive SME management platform.

This phase focuses on implementing the extended requirements **(REQ-020 to REQ-048)**, which encompass Organization Profile Management, Employee & Work Schedule Management, and the Financial Invoicing Module.

To ensure clarity and alignment with the academic methodology, the complete set of functional requirements implemented in the final system is categorized into the following tables:

Table (3.2) Requirements Database RDB

Priority	Code	Requirement	Description
1	REQ-001	Log in	Login for each user level and direct each user to their respective page.
1	REQ-002	Edit personal account	Users can edit their personal data, such as email address and password.
1	REQ-003	View personal account	View user account information.
1	REQ-004	View services list	View a list of services offered by the company.
1	REQ-005	View ratings	View user ratings.
1	REQ-006	View Companies list	View Companies list.
1	REQ-007	Change service provider account status	Change the service provider account status (active/inactive).
1	REQ-008	View users list	View a list of users registered in the system.
1	REQ-009	View categories list	View list of pre-listed categories.
1	REQ-010	Add category	Add a new category to the categories list.
1	REQ-011	Edit category	Edit a pre-existing category.
1	REQ-012	Delete category	Delete a pre-existing category.
2	REQ-013	Export reports	Export system performance reports.

2	REQ-014	View statistics	View comprehensive statistics on system performance.
2	REQ-015	Manage system settings	Managing system settings such as organization name or invoicing mechanism.
2	REQ-016	Add city	Add a new city to the city list.
2	REQ-017	Edit city	Edit an existing city in the city list.
2	REQ-018	Delete city	Delete an existing city from the city list.
2	REQ-019	View cities list	View the list of cities included in the system.
2	REQ-020	Create user accounts	Create new accounts for users who want to use the system.
2	REQ-021	Adding an organization profile	Add an organization profile with name, description and services.
2	REQ-022	Edit an organization profile	Edit a previously added profile.
2	REQ-023	View the profile of the institution	View the previously added organization profile.
2	REQ-024	Add a service	Adding a new service to the list of services provided by the organization.
3	REQ-025	Service modification	Modify previously added service data.
3	REQ-026	Delete a service	Deleting a previously added service.
3	REQ-027	Changing a service status	Change the status of a previously listed service (activation, deactivation).
3	REQ-028	Add an employee	Adding a new employee to the list of employees of the enterprise.
3	REQ-029	Modification of employee data	Modify the data of a previously listed employee.

3	REQ-030	Delete an employee	Delete a previously listed employee.
3	REQ-031	Change employee account status	Change the status of a previously listed employee account (activation, deactivation).
3	REQ-032	View the list of employees	View the list of employees within the organization.
3	REQ-033	Search for an employee	Search for an employee in personnel records.
3	REQ-034	Find a date	Find an appointment in appointment logs.
3	REQ-035	Search for a user	Search for a user in user logs.
3	REQ-036	Search for a service	Search for a service in the service logs.
4	REQ-037	View the appointment list	View the list of appointments booked within the institution.
4	REQ-038	Limit the maximum daily bookings	Determine the maximum number of appointments that the institution receives daily.
4	REQ-039	View Enterprise statistics	View comprehensive statistics on the work of the enterprise.
4	REQ-040	Export Enterprise reports	Export comprehensive reports on the work of the enterprise.
4	REQ-041	View Enterprise ratings	View the ratings provided by customers for the enterprise.
4	REQ-042	Add a payment method	Adding a new payment method to the list of payment methods.
4	REQ-043	Modify a payment method	Modify a previously listed payment method.
4	REQ-044	Delete a payment method	Delete a previously listed payment method.

4	REQ-045	Change the status of a payment method	Change the status of a previously listed payment method (activation, deactivation).
4	REQ-046	View the list of payment methods	View the list of listed payment methods.
4	REQ-047	Determining the work schedule	Determination of working days and working hours of the enterprise.
4	REQ-048	Adjusting the work schedule	Modification of Enterprise Work Data.
5	REQ-049	Change appointment status	Change the status of a pre-booked appointment (acceptance, cancellation).
5	REQ-050	Adding a new invoice	Adding a new invoice by the employee.
5	REQ-051	Modify invoice data	Modify previously added invoice data.
5	REQ-052	Print an invoice	Print a pre-listed invoice.
5	REQ-053	Search for an invoice	Search for an invoice in the billing records.
5	REQ-054	View the list of invoices	View the list of listed invoices.
5	REQ-055	View billing history	View the customer's billing history.
5	REQ-056	Book an appointment	Booking a new appointment by the client.
5	REQ-057	Track appointment status	Track the status of an appointment to find out his status.
5	REQ-058	View appointment history	View the appointment history provided by the client.
5	REQ-059	Add a rating	Add a rating of a service or organization within the system by users.
5	REQ-060	Update Rating	Update a previously added rating.

5	REQ-049	Change appointment status	Change the status of a pre-booked appointment (acceptance, cancellation).
5	REQ-050	Adding a new invoice	Adding a new invoice by the employee.
5	REQ-051	Modify invoice data	Modify previously added invoice data.
5	REQ-052	Print an invoice	Print a pre-listed invoice.
5	REQ-053	Search for an invoice	Search for an invoice in the billing records.
5	REQ-054	View the list of invoices	View the list of listed invoices.
5	REQ-055	View billing history	View the customer's billing history.
5	REQ-056	Book an appointment	Booking a new appointment by the client.
5	REQ-057	Track appointment status	Track the status of an appointment to find out his status.
5	REQ-058	View appointment history	View the appointment history provided by the client.
5	REQ-059	Add a rating	Add a rating of a service or organization within the system by users.
5	REQ-060	Update Rating	Update a previously added rating.

3.5 Feasibility and Risks:

Feasibility Study

1. **Technical Feasibility:** The project relies on mature, open-source technologies (Python/Django, SQLite, Bootstrap). The development team possesses the requisite skills in backend logic, database design, and frontend RTL implementation, ensuring all technical challenges can be resolved.
2. **Operational Feasibility:** The system directly addresses the operational bottlenecks of small businesses. By providing an intuitive, Arabic-native interface, it ensures high adoption rates among non-technical staff and business owners.
3. **Economic Feasibility:** As an open-source solution, the system eliminates recurring SaaS subscription fees. The only costs incurred are minimal server hosting or local maintenance, offering a massive Return on Investment (ROI) for SMEs.
4. **Academic Feasibility:** The project comprehensively covers multiple domains in Software Engineering and Information Systems, including:
    - Requirements engineering and system analysis.
    - Relational database design and optimization.
    - Web development (Backend & Frontend).
    - Software testing and quality assurance.
    - Project management and agile methodologies.

Risks and Mitigation Plans

Table (3.3) Risk Management Matrix

Risk	Impact	Probability	Mitigation Plan
Complex Scheduling Logic: Difficulty in preventing conflicts when multiple employees and work schedules are involved.	High	Medium	Implement rigorous backend validation algorithms using Django ORM. Conduct early prototyping and unit testing for the booking engine.
Data Integrity in Invoicing: Potential for orphaned records or incorrect financial calculations when linking invoices to appointments.	High	Low	Enforce strict foreign key constraints (CASCADE/PROTECT) in the ERD. Utilize database transactions to ensure atomic operations.
Scope Creep: Addition of unplanned features (e.g., AI integration, external payment gateways) delaying the academic deadline.	Medium	High	Strictly adhere to the defined scope (REQ-020 to REQ-048). Document advanced features as "Future Work" to maintain focus.
RTL UI Breaking: Complex data tables (like Invoices or Employee lists) breaking the Arabic RTL layout on mobile devices.	Medium	Medium	Utilize Bootstrap 5 RTL grid system exclusively. Conduct continuous cross-device responsive testing during the UI phase.

3.6 Project Plan:

The project execution is structured around the Incremental Methodology, visualized via a Gantt Chart (to be inserted as Figure 3.1 in the final document). The plan is divided into the following key phases and milestones:

1. **Phase 1: Foundation & Core Admin (Completed - Increment 1)**
    - *Duration:* Weeks 1-4
    - *Milestone:* Completion of REQ-001 to REQ-019 (Authentication, Cities, Categories, Basic Providers).
2. **Phase 2: Enterprise & HR Expansion (Current Phase - Increment 2)**
    - *Duration:* Weeks 5-8
    - *Milestone:* Implementation of REQ-020 to REQ-038 (Organization Profiles, Employee Management, Work Schedules, Advanced Booking Logic).
3. **Phase 3: Financial Module & Analytics (Current Phase - Increment 2)**
    - *Duration:* Weeks 9-11
    - *Milestone:* Implementation of REQ-039 to REQ-048 (Invoicing, Payment Methods, Statistics, Reports).
4. **Phase 4: Integration Testing & Refinement**
    - *Duration:* Week 12
    - *Milestone:* System-wide testing, bug fixing, and RTL UI polishing.
5. **Phase 5: Documentation & Final Submission**
    - *Duration:* Weeks 13-14
    - *Milestone:* Finalizing the academic report, preparing the presentation, and project defense.

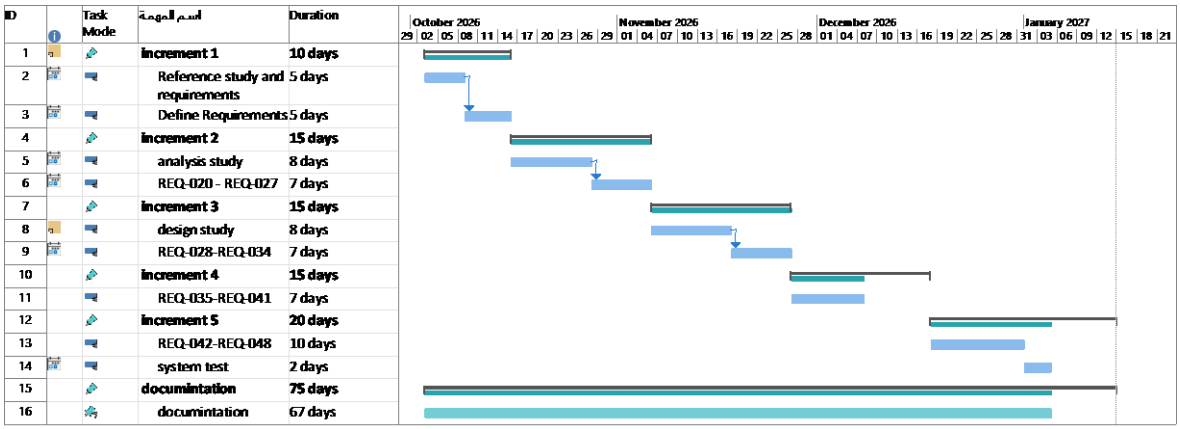


Figure (3.1) Gantt Chart

Task Distribution:

- **Shahd Hilal:** Backend architecture, Database design (ERD), Implementation of Employee & Scheduling logic (REQ-028 to REQ-038), and Invoicing module.
- **Firas Al-Zoubi:** Frontend development (Bootstrap RTL), UI/UX design, Implementation of Organization Profiles, Payment Methods, and Reporting dashboards.

Chapter 4
Analytical and Design Study

4.1 High-Level Architecture:

The system adopts a **Layered Architecture** combined with the **Model-View-Template (MVT)** design pattern native to the Django framework. This separation of concerns ensures modularity and ease of maintenance:

1. **Presentation Layer (Templates):** Bootstrap 5 (RTL) HTML templates that handle the user interface, ensuring responsive design and native Arabic localization.
2. **Application Logic Layer (Views & URLs):** Django Views process HTTP requests, enforce Role-Based Access Control (RBAC), and execute business logic (e.g., schedule conflict resolution). URL routers (urls.py) map endpoints to specific views.
3. **Data Access Layer (Models & ORM):** Django's Object-Relational Mapper abstracts database operations, ensuring data integrity and preventing SQL injection.
4. **Persistence Layer (Database):** An SQLite database stores structured relational data, including organizational profiles, employee schedules, and financial invoices.

4.2 Use Case Diagram:

The Use Case Diagram illustrates the interactions between the four primary actors (System Admin, Service Provider/Owner, Employee, Customer).

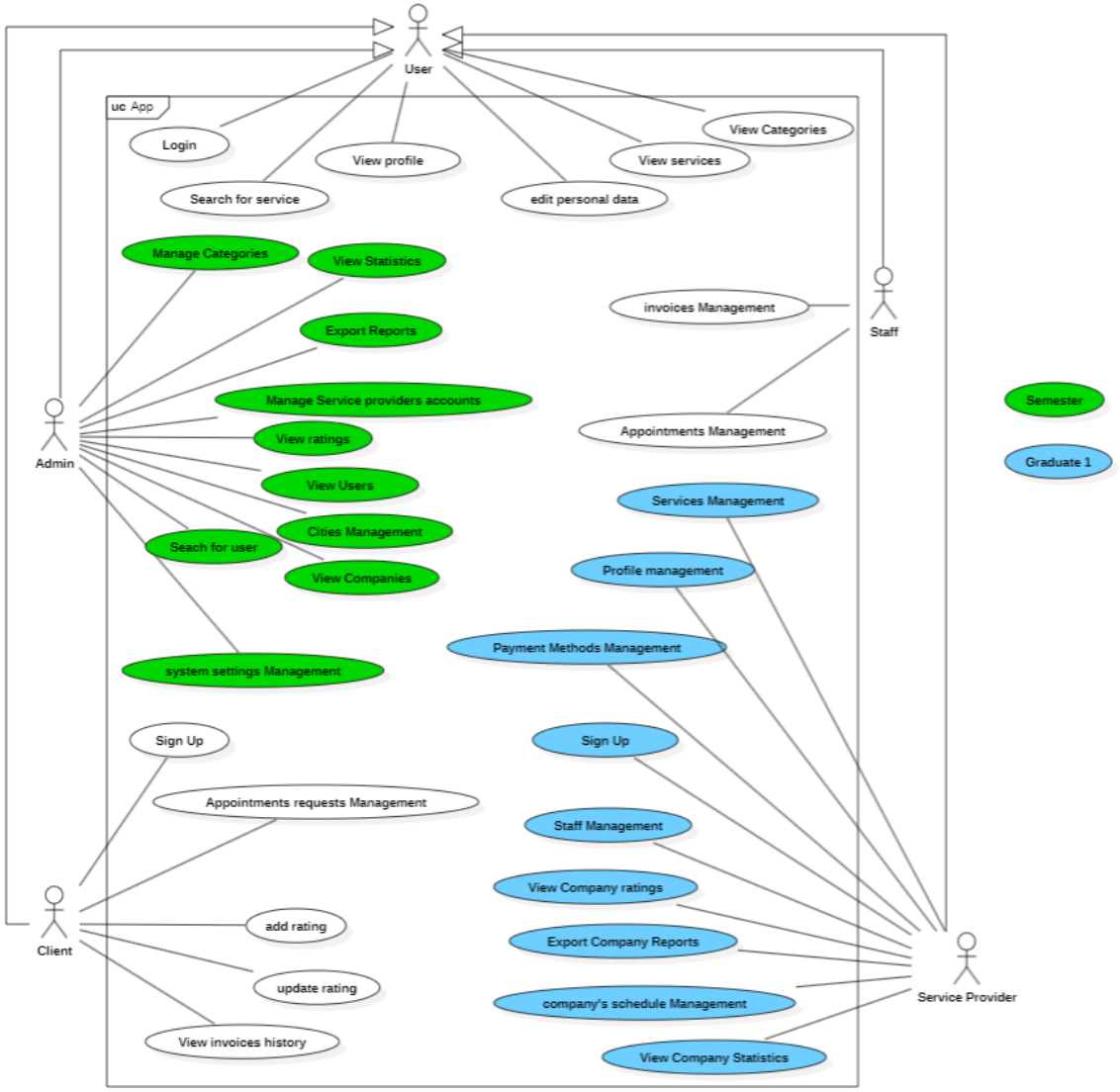


Figure (4.1) Use Case Diagram

4.2.1 Use Case Descriptions:

Below are detailed descriptions for five critical use cases representing the core and extended system functionalities:

Table (4.1) Set & Adjust Work Schedule use case

Use Case ID	**UC-04**
Use Case	Set & Adjust Work Schedule
Description	Configures the operational hours, break times, and holidays for the enterprise, enabling the booking engine to accurately calculate and restrict available appointment slots
Actors	Service Provider (Owner)
Pre-Conditions	- The Owner is authenticated. - The enterprise has an active Company profile
Subsequent-Conditions	- The Working Hour records are saved/updated in the database. - The booking engine restricts customer appointments to these defined operational hours and excludes break times.
Main Flow	1. Owner navigates to the "Work Schedule" module (/owner-working-hours/). 2. System retrieves and displays the current WorkingHour configurations for each day of the week. 3. Owner selects a specific day and toggles the "Off/Holiday" status (is_off) or enters Start Time, End Time, and Break intervals. 4. System validates the time logic (e.g., End Time > Start Time, Break Time within working hours). 5. System saves or updates the WorkingHour record linked to the Company in the database.
Alternative Flow	**A1: Invalid Time Range** 4a. If the End Time is earlier than the Start Time, the system displays a validation error ("Invalid time range") and prevents saving

Table (4.2) Manage Payment Method use case

Use Case ID	**UC-05**
Use Case	Manage Payment Method
Description	Allows the enterprise owner to configure, activate, deactivate, or remove the accepted payment methods (e.g., Cash, Bank Transfer, Mobile Payment) available for financial transactions and invoicing.
Actors	Service Provider (Owner)
Pre-Conditions	- The Owner is authenticated. - The Owner has access to the financial settings module.
Subsequent-Conditions	- The Payment Method records are updated in the database, reflecting the newly added, modified, or toggled payment options for the enterprise.
Main Flow	1. Owner navigates to the "Payment Methods" management page (/owner-payment-methods/). 2. System displays the current list of configured payment methods and their active statuses. 3. Owner selects an action (Add, Edit, or Toggle Status) for a payment method. 4. If adding/editing, Owner enters the Method Name, Arabic Name, Payment Type (cash, credit_card, etc.), and Description. 5. System validates the input format and checks for constraints. 6. System creates, updates, or toggles the is_active boolean flag of the PaymentMethod record in the database.
Alternative Flow	**A1: Invalid Payment Type Selection** 5a. If the Owner fails to select a valid payment_type from the predefined choices, the system rejects the form submission and highlights the missing field.

Table (4.3) Manage Employees & HR use case

Use Case ID	UC-09
Use Case	Manage Employees & HR
Description	Allows the Service Provider (Owner) to add new employees to the enterprise's personnel registry, modify their professional data, toggle their active/inactive status, and search for employees, fulfilling requirements REQ-028 to REQ-033.
Actors	Service Provider (Owner)
Pre-Conditions	- The Owner is authenticated. - The enterprise has an active Company profile. - The employee to be added has a registered User account in the system.
Subsequent-Conditions	- The Employee records are created, updated, or their status is toggled in the database. - The employee becomes available for assignment to services, work schedules, or appointments.
Main Flow	1. Owner navigates to the "Employees" module (/owner-employees/). 2. System retrieves and displays the current list of employees linked to the Company. 3. Owner selects an action (Add, Edit, Toggle Status, or Search). 4. If adding, Owner enters employee details (Role, Contact Info) and links them to an existing User account. 5. System validates the input format and checks for constraints (e.g., ensuring the user is not already an employee in another company). 6. System creates or updates the Employee record linked to the Company in the database.
Alternative Flow	**A1: Invalid Employee Data or User Link** 5a. If the linked User account does not exist, or if the user is already registered as an employee elsewhere, the system rejects the submission and displays a validation error ("Invalid User" or "User already assigned").

Table (4.4) Manage Organization Profile & Services use case

Use Case ID	**UC-03**
Use Case	Manage Organization Profile & Services
Description	Enables the service provider to define and maintain the enterprise's public profile and the catalog of services offered, including pricing, duration, and active status.
Actors	Service Provider (Owner)
Pre-Conditions	- The Owner is authenticated. - The Owner has an active user account
Subsequent-Conditions	- The Company profile or Service records are successfully created or updated in the database, making them visible to customers for browsing and booking.
Main Flow	1. Owner accesses the "Services Catalog" module (/owner-services/) from the dashboard. 2. System retrieves and displays the list of existing services. 3. Owner clicks "Add New Service". 4. Owner enters the required details (e.g., Service Name, Arabic Name, Description, Duration in minutes, Price). 5. System validates the input (e.g., Price > 0, Duration > 0). 6. System creates the Service record linked to the Company in the database.
Alternative Flow	**A1: Invalid Pricing or Duration** 5a. If the Price or Duration is entered as zero or a negative number, the system rejects the submission and displays "Price and Duration must be greater than zero."

Table (4.5) Manage Global Entities use case

Use Case ID	**UC-01**
Use Case	Manage Global Entities
Description	Allows the System Administrator to maintain the foundational lookup tables, specifically Cities and Categories, which standardize location and service classification across the entire platform.
Actors	System Administrator
Pre-Conditions	- The Administrator is authenticated with superuser privileges.
Subsequent-Conditions	- The City or Category records are added, modified, or deleted in the database, instantly updating the global dropdowns available to providers and customers.
Main Flow	1. Administrator navigates to the "Cities" or "Categories" management section in the Admin Dashboard (/dashboard-admin/cities/ or /categories/). 2. System displays the current list of global entities. 3. Administrator selects an action (Add, Edit, or Delete) for a specific entity. 4. If adding/editing, Administrator enters the Name and Arabic Name (and Description for Categories). 5. System validates the input and checks for uniqueness constraints. 6. System executes the Create, Update, or Delete operation on the respective model in the database.
Alternative Flow	**A1: Deletion Constraint Violation** 6a. If the Administrator attempts to delete a City or Category that is currently linked to active Company or Service records, the system blocks the deletion (via PROTECT or custom logic) and displays "Cannot delete: Entity is currently in use by active providers."

4.3 Activity Diagrams:

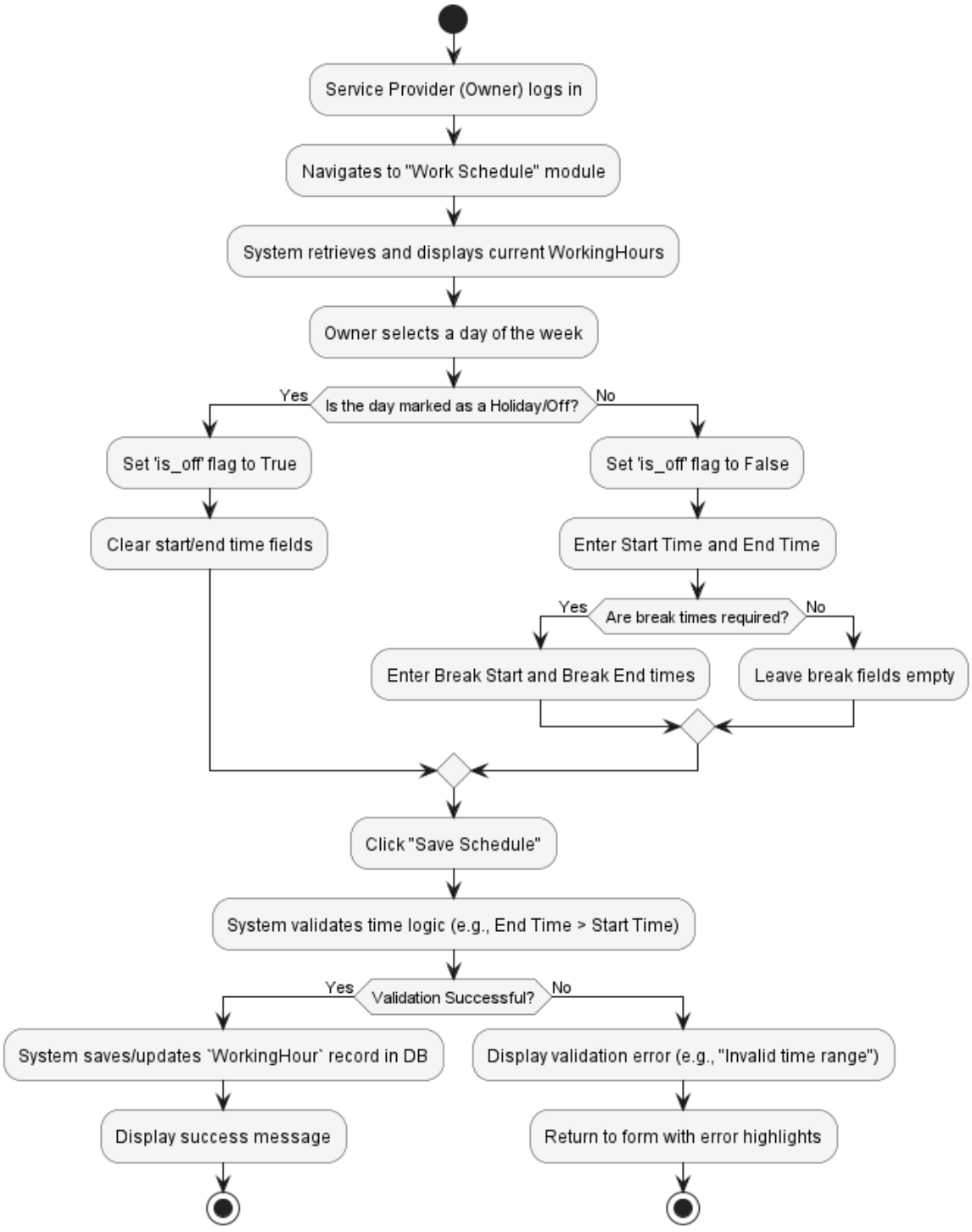


Figure (4.2) Activity Diagram for Set & Adjust Work Schedule

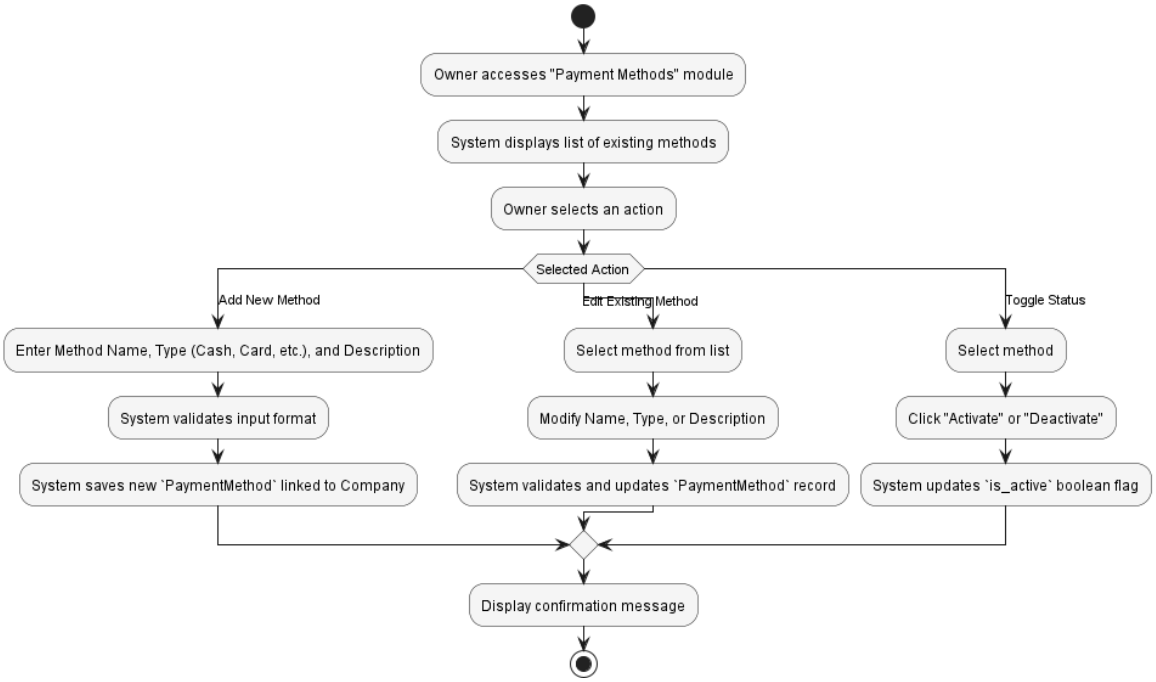


Figure (4.3) Activity Diagram for Payment Methods Management

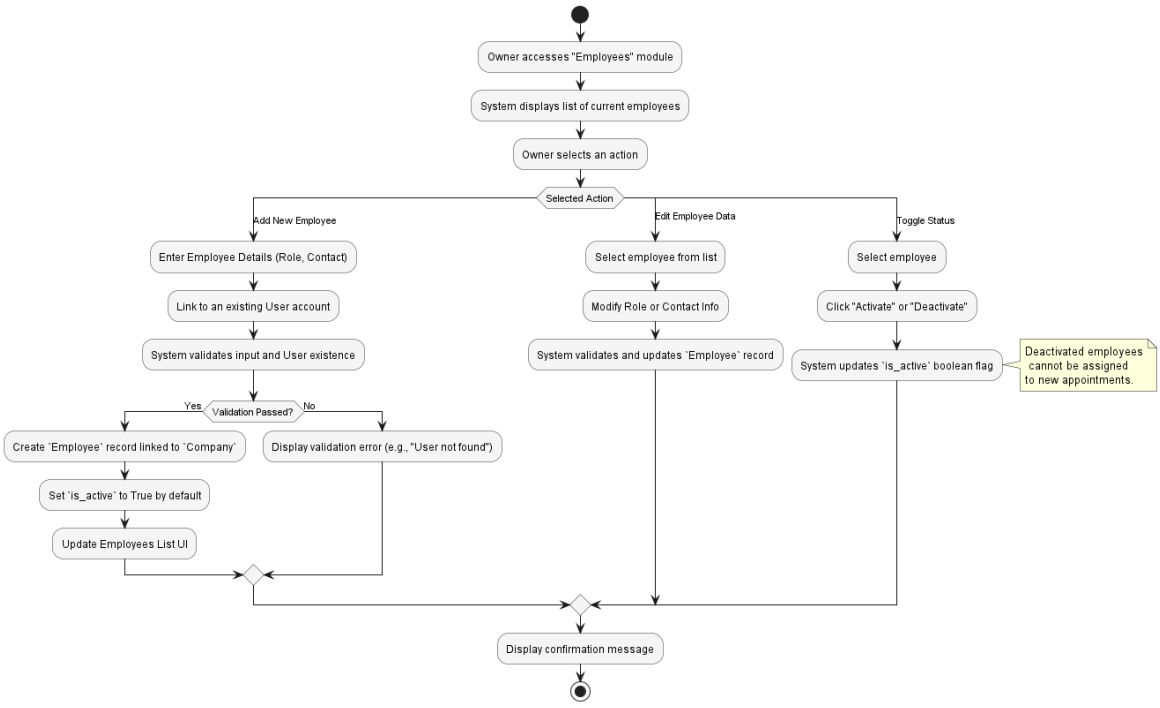


Figure (4.4) Activity Diagram for Manage Employees & HR

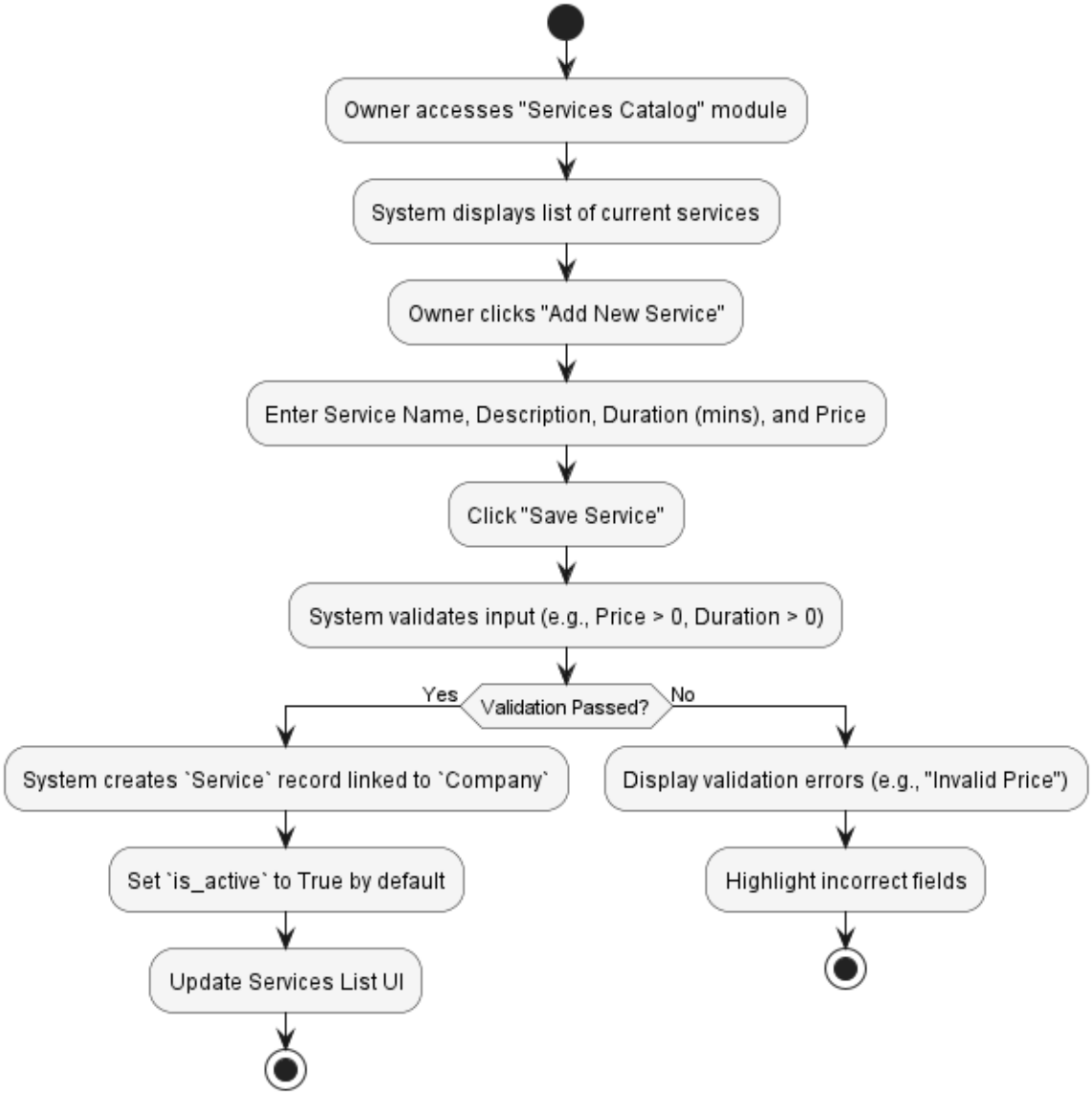


Figure (4.5) Activity Diagram for Manage Organization Profile & Services

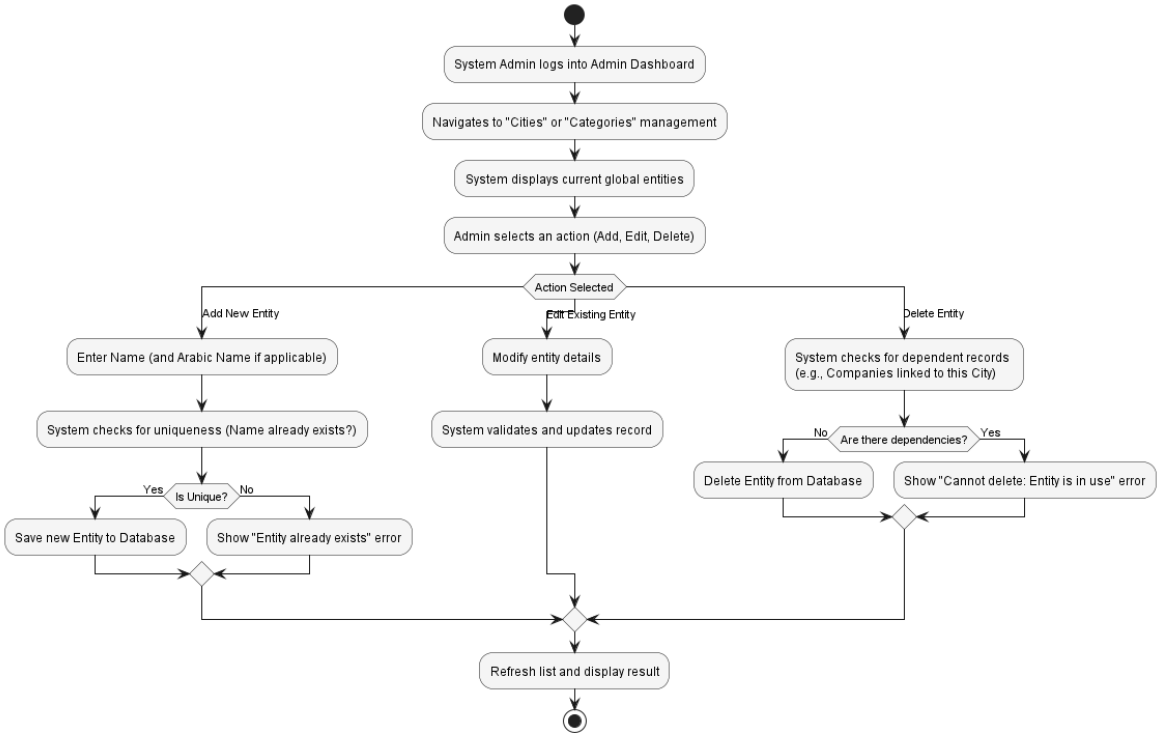


Figure (4.5) Activity Diagram for Manage Global Entities

4.4 Sequence diagrams:

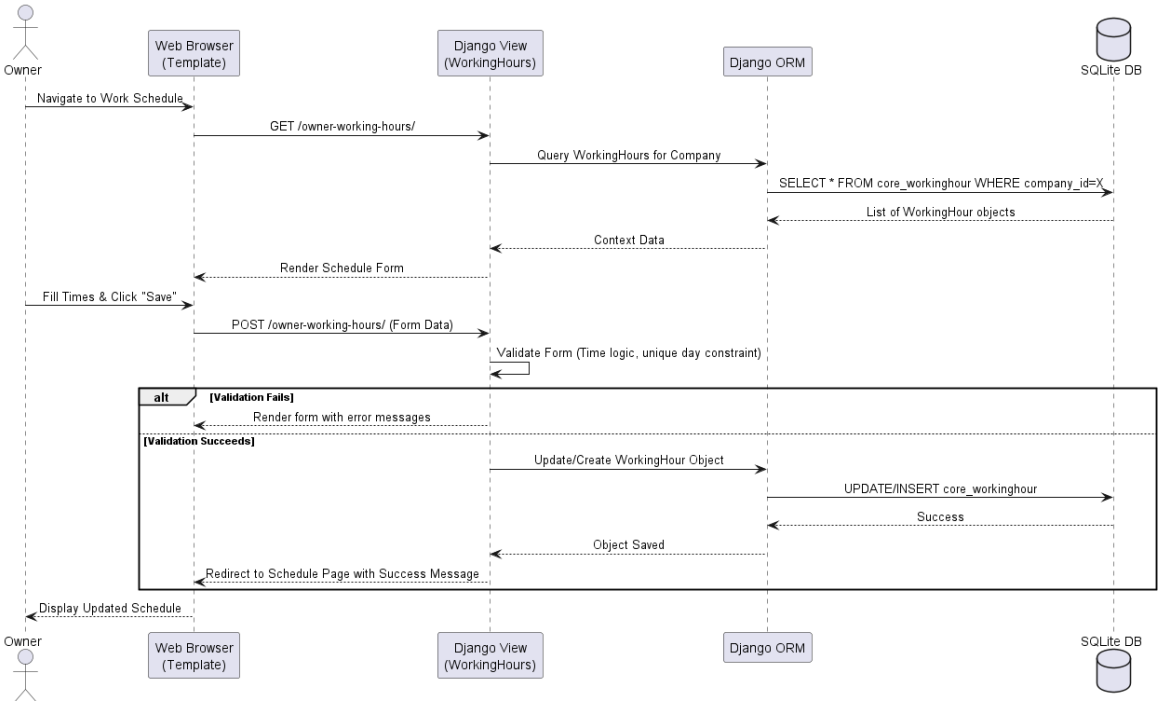


Figure (4.6) Sequence Diagram for Set & Adjust Work Schedule

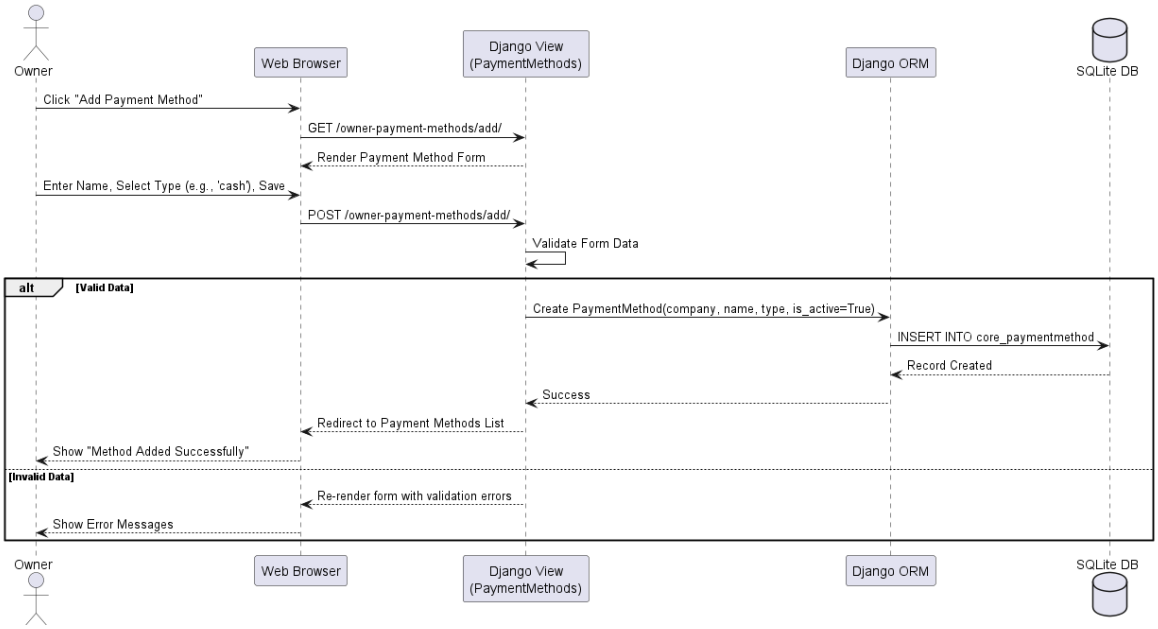


Figure (4.7) Sequence Diagram for Payment Methods Management

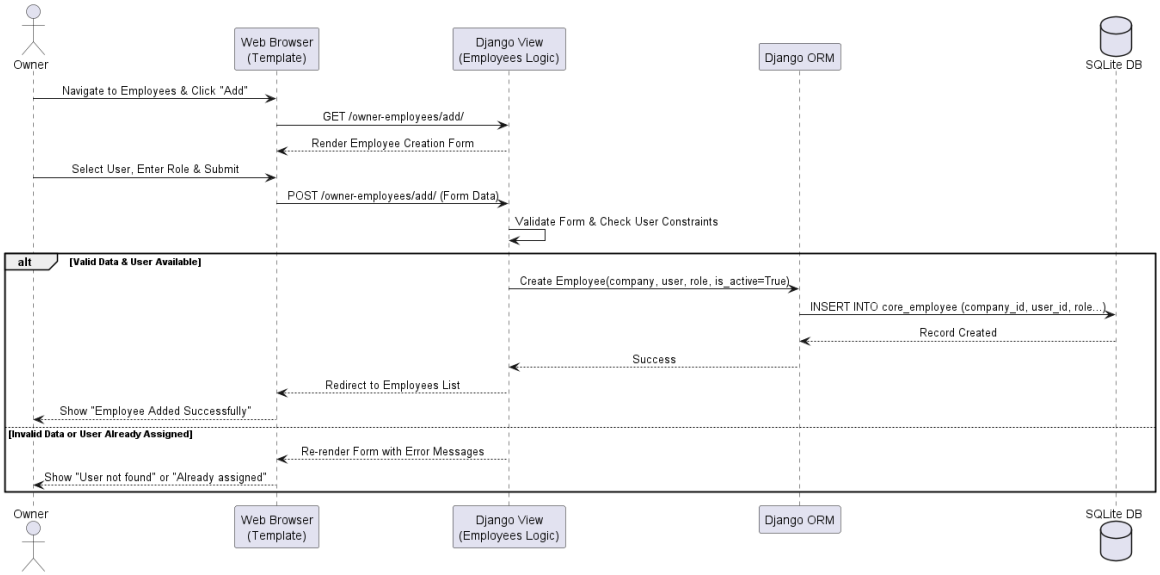


Figure (4.8) Sequence Diagram for Manage Employees & HR

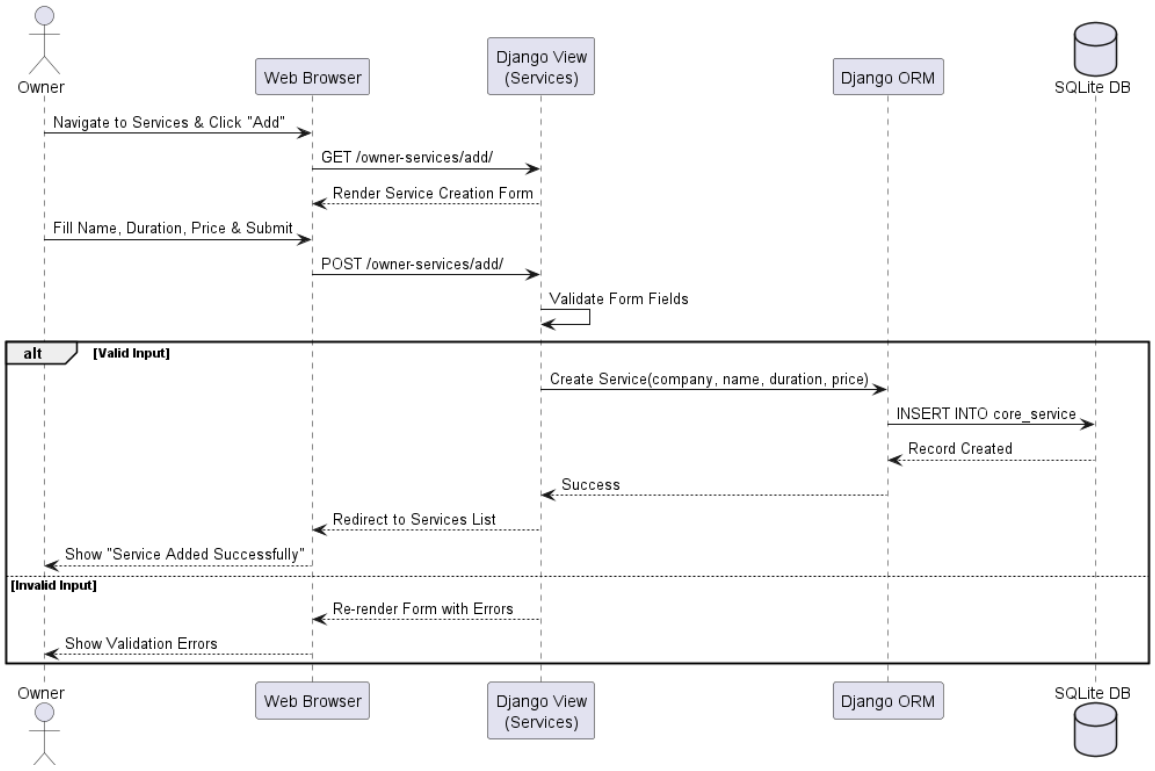


Figure (4.9) Sequence Diagram for Manage Organization Profile & Services

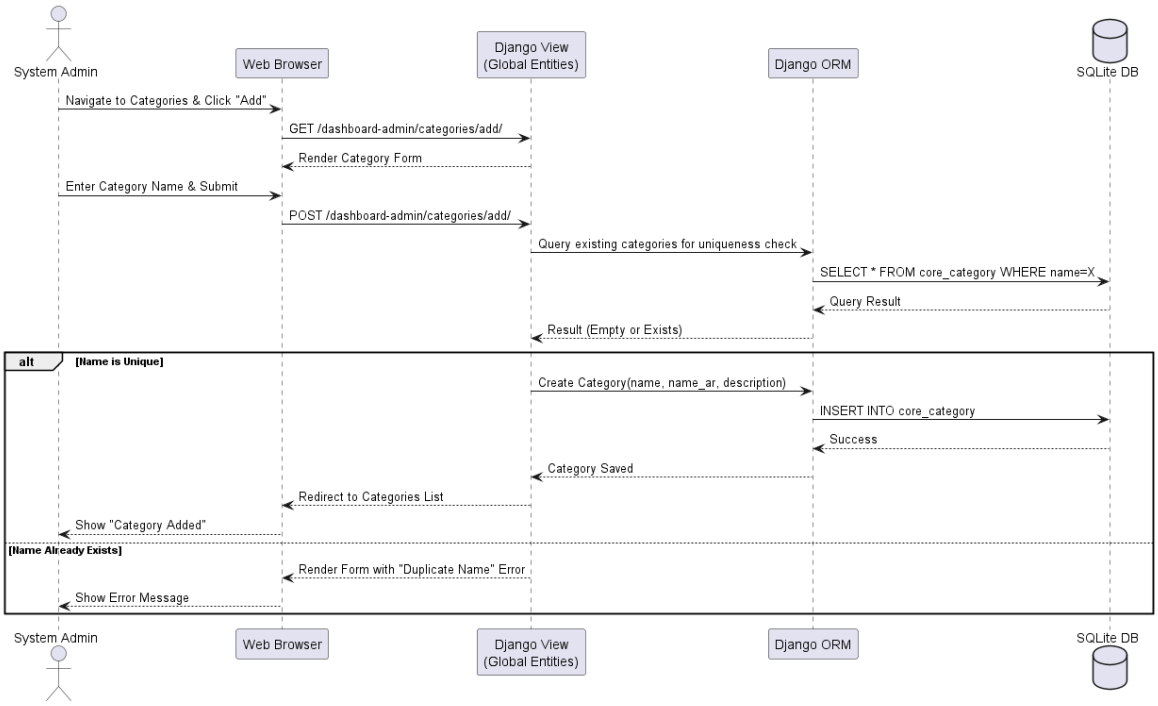


Figure (4.10) Sequence Diagram for Manage Global Entities

4.5 Class Diagram:

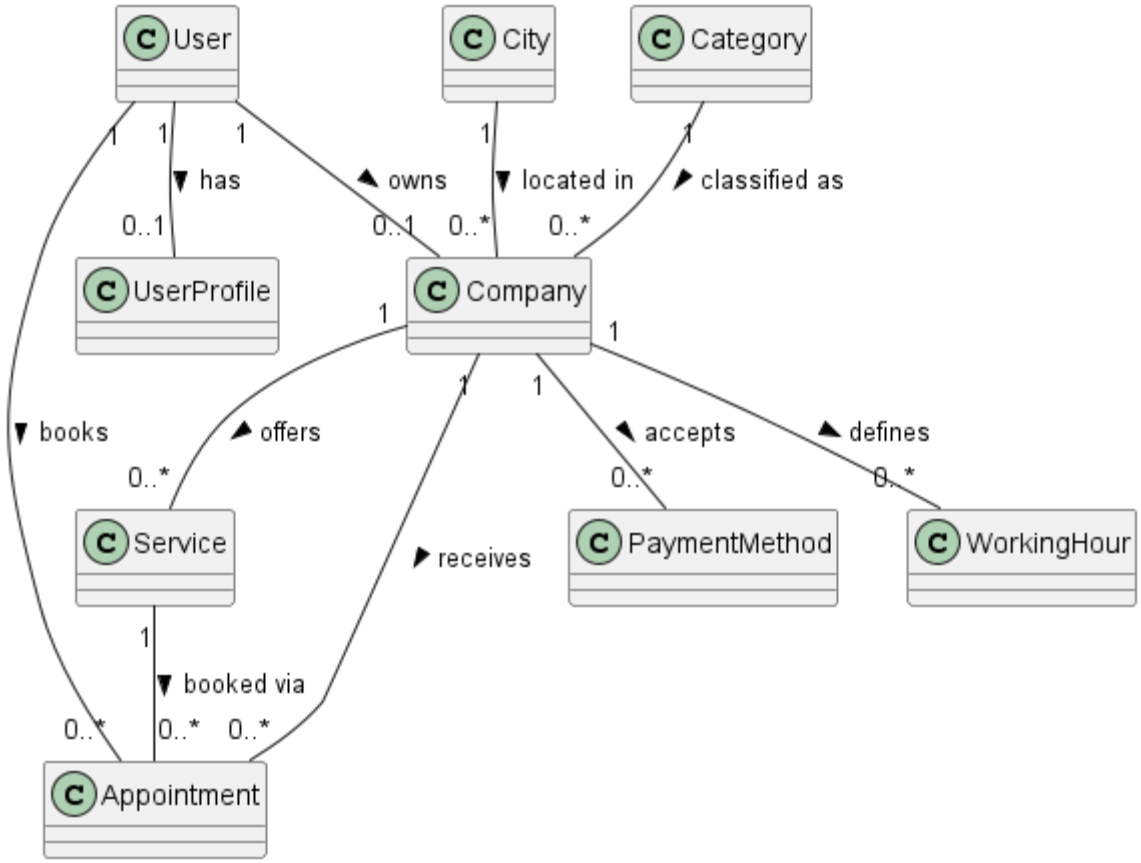


Figure (4.11) Class Diagram

4.6 Component Diagram:

The Component Diagram illustrates the physical and logical deployment architecture of the web application, demonstrating the separation of the Client, Application, and Data tiers.

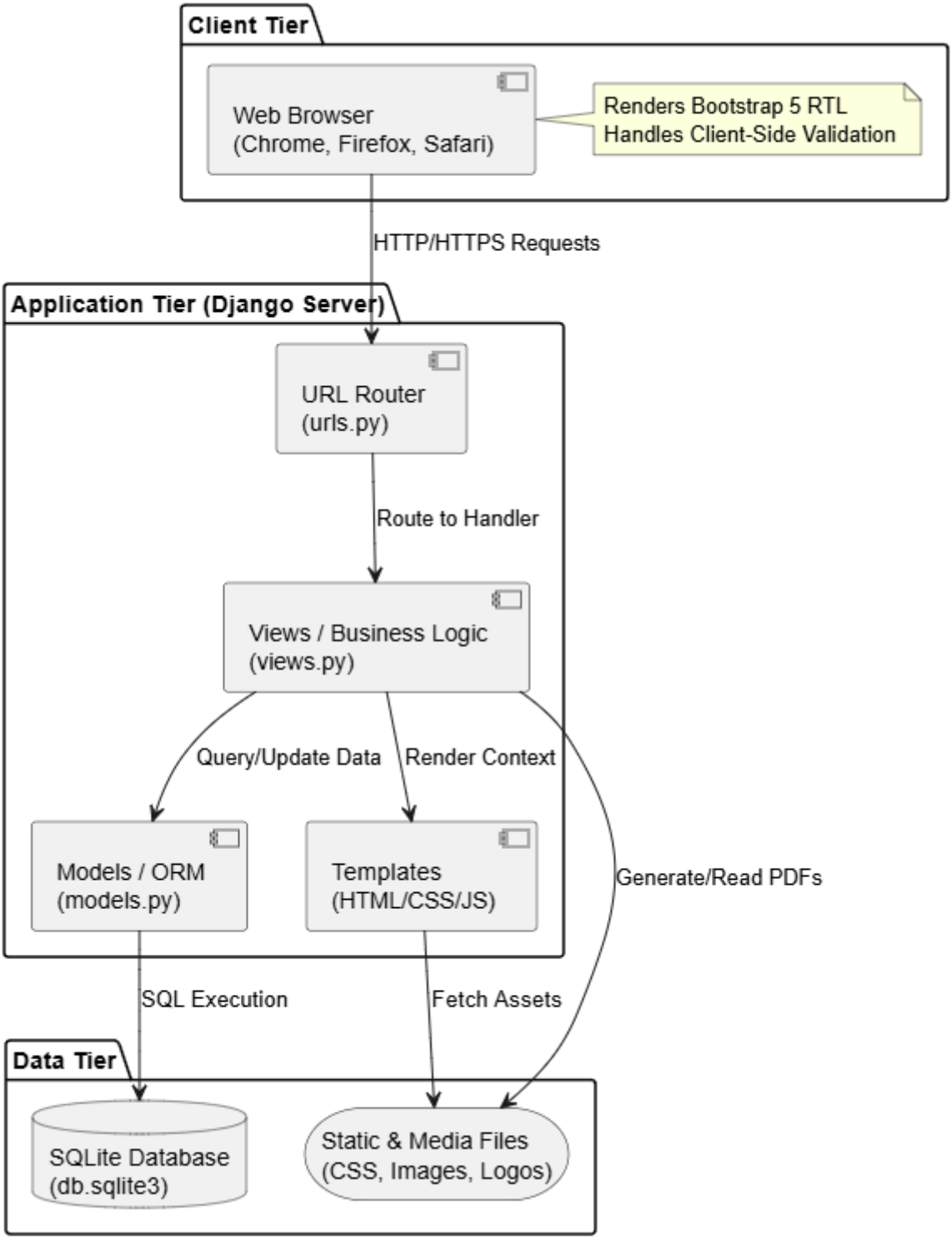


Figure (4.12) Component Diagram of the System Architecture

4.7 Entity-Relationship Diagram (ERD):

The ERD maps the relational schema of the SQLite database, highlighting primary keys (PK), foreign keys (FK), and cardinalities.

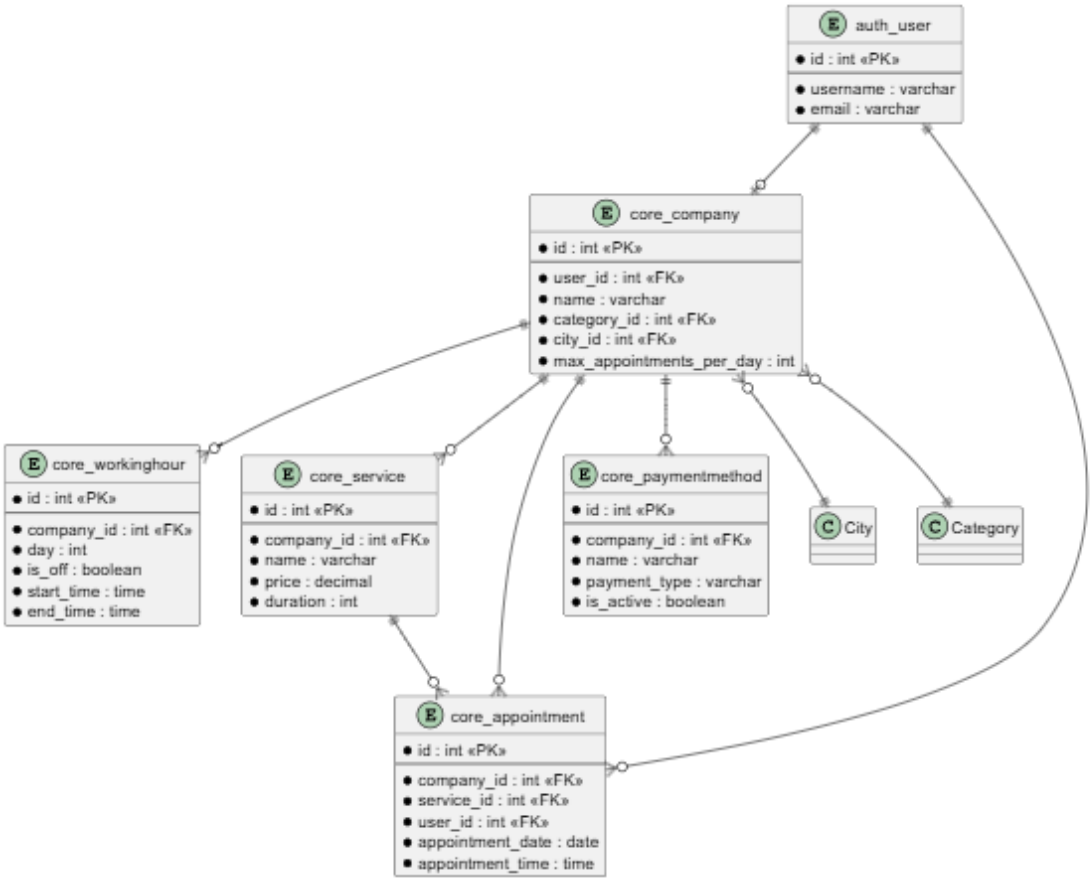


Figure (4.13) Entity-Relationship Diagram (ERD)

4.8 Sitemap:

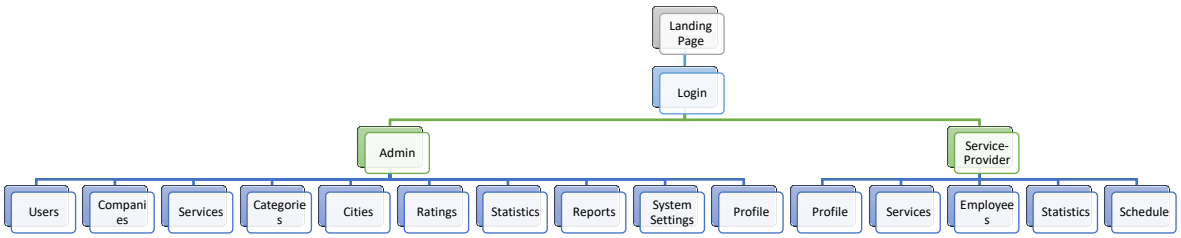


Figure (4.14) Sitemap

4.9 Initial Tests (Unit Testing):

To ensure the integrity of the models and business logic defined in models.py, initial unit tests were designed using Django's TestCase framework.

Table (4.6) Initial Unit Test Cases

Test ID	Target Model / Function	Test Description	Expected Outcome
UT-01	Company.is_open_on()	Verify if the system correctly identifies a holiday (is_off=True) as "Closed".	Returns False.
UT-02	Company.get_available_slots()	Test if the system correctly skips break_start to break_end times when generating slots.	Break times are excluded from the returned list.

UT-03	Appointment.save()	Verify that end_time is automatically calculated based on service.duration if not provided.	end_time equals appointment_time + duration.
UT-04	WorkingHour	Ensure a company cannot have two overlapping schedules for the same day (unique_together).	IntegrityError is raised upon duplicate save.
UT-05	PaymentMethod	Validate that a payment method is strictly linked to one company (Foreign Key constraint).	Deletion of company cascades/protects correctly.

4.10 Algorithmic Design: Conflict Resolution:

A core algorithmic component of the system is the Automated Conflict Resolution logic, implemented within the Company.get_available_slots(date) method in models.py. This algorithm ensures that customer bookings never overlap with existing appointments, breaks, or non-working hours.

```
FUNCTION get_available_slots(company, target_date):
    working_hours = GET WorkingHour WHERE company=company AND
day=target_date.weekday()
    IF working_hours.is_off OR working_hours IS NULL:
        RETURN Empty List

    current_time = working_hours.start_time
    end_time = working_hours.end_time
    available_slots = []

    WHILE current_time < end_time:
        // 1. Skip Break Times
        IF working_hours.break_start AND working_hours.break_end:
            IF current_time >= working_hours.break_start AND current_time <
working_hours.break_end:
```

```
                current_time = working_hours.break_end
                CONTINUE

        // 2. Check for Existing Appointments
        is_booked = EXISTS Appointment WHERE company=company
                    AND appointment_date=target_date
                    AND appointment_time=current_time
                    AND status IN ('pending', 'confirmed')

        // 3. Check Daily Max Limit
        daily_count = COUNT Appointment WHERE company=company AND
appointment_date=target_date
        IF daily_count >= company.max_appointments_per_day:
            BREAK // Stop searching, limit reached

        IF NOT is_booked:
            available_slots.APPEND(current_time)

        // 4. Increment by 15 minutes
        current_time = current_time + 15 minutes

    RETURN available_slots
```

4.11 Requirements Traceability Matrix (RTM):

The RTM ensures that every functional requirement is traced through design, implementation, and testing, guaranteeing complete coverage of the system's scope.

Table (4.7) Requirements Traceability Matrix

Req ID	Requirement Title	Use Case	Design Artifact (Class/ERD)	Implementation (URLs / Models)	Test Case
REQ-001	Log in	UC-Auth	User, UserProfile	urls.py: /login/ models.py: User	TC-01
REQ-010	Add category	UC-01	Category	urls.py: /dashboard-admin/categories/add/ models.py: Category	TC-10

REQ-021	Add Organization Profile	UC-03	Company, City, Category	urls.py: /owner-company/edit/ models.py: Company	TC-21
REQ-028	Add an employee	UC-07	Employee, Company	urls.py: /owner-employees/add/ models.py: Employee	TC-28
REQ-042	Add payment method	UC-05	PaymentMethod, Company	urls.py: /owner-payment-methods/add/ models.py: PaymentMethod	TC-42
REQ-047	Set work schedule	UC-04	WorkingHour, Company	urls.py: /owner-working-hours/ models.py: WorkingHour	TC-47

Chapter 5
Implementation

5.1 Execution Environment:

The development and testing of the system were conducted in a controlled environment to ensure consistency and reproducibility. The execution environment is divided into hardware and software specifications:

1. Hardware Specifications:

- **Processor:** Intel Core i5 / i7 (or equivalent AMD Ryzen).
- **RAM:** 8 GB or higher (to support simultaneous execution of the IDE, database browser, and local server).
- **Storage:** 256 GB SSD (for fast read/write operations during database migrations and static file collection).

2. Software Specifications:

- **Operating System:** Windows 10/11 (64-bit).
- **Programming Language:** Python 3.10+.
- **Web Framework:** Django 4.2.x.
- **Database Engine:** SQLite 3.
- **Integrated Development Environment (IDE):** Visual Studio Code (equipped with Python, Django, and Prettier extensions).
- **Database Management Tool:** DB Browser for SQLite.
- **Version Control:** Git and GitHub.
- **Web Browsers:** Google Chrome and Mozilla Firefox (for cross-browser compatibility and RTL rendering testing).

5.2 Software Architecture of the Project:

The project follows a modular and scalable directory structure based on Django's Model-View-Template (MVT) architectural pattern. The root directory contains the following critical components:

1. **manage.py:** The command-line utility used for administrative tasks, such as running the local server (runserver), applying database migrations (migrate), and creating superusers.
2. **db.sqlite3:** The lightweight relational database file that stores all system entities, including users, companies, services, and appointments.
3. **requirements.txt:** A manifest file listing all Python dependencies (e.g., Django, Pillow, reportlab, openpyxl) to ensure consistent environment setup.
4. **core/ (Main Application Package):** Encapsulates the primary business logic. It is subdivided into:
    - **models.py:** Defines the database schema and business entities (e.g., Company, WorkingHour, Service, PaymentMethod).
    - **views.py:** Contains the Python functions that process HTTP requests, execute business logic, and return HTTP responses.
    - **urls.py:** Maps URL patterns to their corresponding view functions.
    - **templates/:** Houses the HTML files, utilizing Django's template language to render dynamic data passed from the views.
5. **static/ and media/:** Directories for storing CSS, JavaScript, images, and user-uploaded files (e.g., company logos).